Despliegue en la nube de un modelo de mora de créditos: Google Cloud y AWS

Ciencia de Datos en Producción, Universidad Pontificia Bolivariana. 11 de noviembre de 2026.
Nombre: _____ Código: _____
Repositorio: github.com/rosvend/MLOps. Documentación: https://rosvend.github.io/MLOps/

1. Introducción y arquitectura

Este informe cuenta cómo se llevó a la nube un modelo que predice la mora de 10.763 créditos (4,75 % en mora). El modelo y sus variables vienen de la entrega anterior; aquí importa la infraestructura. Todo corre en un proyecto de Google Cloud (`us-central1`): entrenamiento en Vertex AI, una API en Cloud Run, registro de modelos, log de predicciones en BigQuery y cuatro workflows de GitHub Actions para pruebas, despliegue, reentrenamiento y detección de deriva. Se usó GitHub Actions porque el código ya está en GitHub y así no hay un servidor que mantener. La misma API se desplegó también en AWS para comparar.

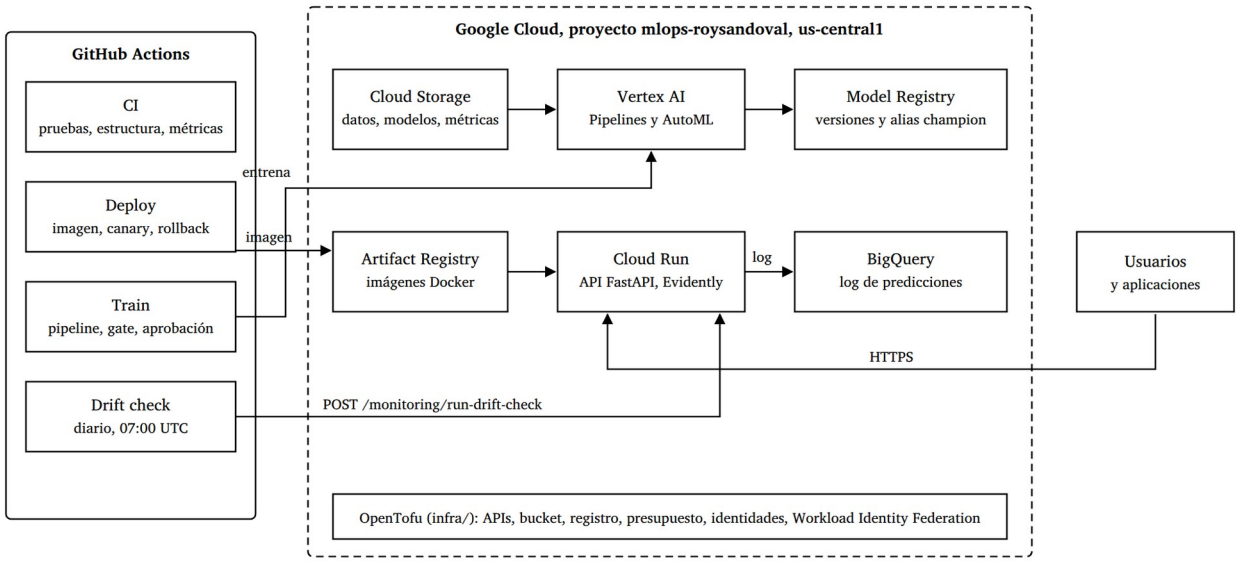


Figura 1. Arquitectura en Google Cloud. Dibujo propio a partir de `docs/gcp/overview.md`.

2. Infraestructura en Google Cloud

La infraestructura está escrita con OpenTofu en `infra/`: las APIs, un bucket privado, el repositorio de imágenes en Artifact Registry, una alerta de presupuesto al 50 % y al 90 % de 100 USD (avisa, no frena el gasto) y las identidades. GitHub entra con Workload Identity Federation: el workflow cambia un token de corta vida por una credencial y una condición limita el acceso al repositorio `rosvend/MLOps`, así que no hay llaves guardadas. Hay tres cuentas de servicio con permisos mínimos: `github-ci` (escribe en un repositorio de imágenes, despliega en Cloud Run sin poder cambiar políticas IAM y lee un bucket), `scoring-api` (corre la API y escribe en un dataset de BigQuery) y `vertex-pipeline` (corre el pipeline de entrenamiento).

El montaje dio tres tropiezos. El presupuesto creado justo después de habilitar su API falló con `SERVICE_DISABLED` y se repitió `apply`. Las credenciales locales traían el quota project de otro proyecto. Y el agente de servicio de Vertex necesitó permiso de lectura sobre el registro de imágenes antes de que corriera el primer pipeline.

3. Despliegue y CI/CD

Tabla 1. Workflows de GitHub Actions.

Workflow	Se dispara con	Qué hace
CI	Pull request y push a `main`	Clona el repositorio, instala dependencias (`uv sync`), valida la estructura de carpetas (`tests/test_repo_structure.py`), valida las métricas del campeón (PR-AUC mínimo 0,12 y Gini mínimo 0,30; el campeón tiene 0,1293 y 0,3272) y corre `pytest`. El job `notify` corre siempre: escribe el resumen y comenta el commit, y GitHub envía ese comentario por correo.
Deploy	Push a `main` que toque el servicio, a mano, o llamado por Train	Construye la imagen (etiqueta = SHA del commit), crea una revisión de Cloud Run sin tráfico, prueba `/health` y una predicción, pasa el 10 %, analiza con métricas, pasa el 100 % o hace rollback.
Train	Lunes 06:00 UTC, cambios en el código de entrenamiento, deriva o a mano	Entrena tres candidatos en Vertex AI, compara el mejor con el campeón (puerta), pide aprobación humana, promueve y lanza Deploy.

Workflow	Se dispara con	Qué hace
Drift check	Cada día a las 07:00 UTC	Llama a la API privada con un token de identidad y a `POST /monitoring/run-drift-check`; si hay deriva, lanza Train.

Puerta y aprobación. La puerta promueve un candidato solo si su PR-AUC sube al menos 0,005 y el intervalo bootstrap pareado de la diferencia queda sobre cero. Si promueve, el job `promote` se detiene en el entorno `production` hasta que el revisor aprueba; después copia los archivos del campeón, registra una versión nueva en el Model Registry, mueve el alias `champion` y lanza Deploy. Train corrió dos veces: en una la puerta respondió «keep» (candidato y campeón son el mismo modelo, PR-AUC 0,1293) y en otra, con `force_promote`, se ejercitó la ruta completa y quedó registrada la versión 2 de `loan-default-scorer`. Esa aprobación la hizo el asistente de IA a petición del revisor. Nunca hubo una decisión «promote» real, y la puerta evalúa sobre meses de prueba que ya se usaron para elegir modelo.

Canary con métricas. Deploy crea la revisión sin tráfico y la prueba por su URL con etiqueta. Con el 10 % del tráfico envía 40 solicitudes de prueba a cada revisión durante 2,5 minutos, espera dos minutos a que Cloud Monitoring se ponga al día y lee de cada revisión el número de solicitudes, la proporción de errores 5xx y el p95 de latencia. Hace rollback si la revisión nueva tiene menos de 20 solicitudes en las métricas, más de 2 % de errores 5xx, un p95 sobre 6 s o sobre el doble del de la anterior, o si falla una prueba; los límites están en `config/ci/canary.yaml`. Se ejecutó tres veces el 12 de noviembre. La primera falló por un error nuestro de etiquetado y revirtió con «solo 4 solicitudes en las métricas». Una ejecución normal pasó: la revisión nueva tuvo 45 solicitudes, 0 % de 5xx y p95 de 153 ms, contra 41 solicitudes y 308 ms de la anterior. En la tercera se fijó el tope de latencia en 1 ms a propósito y falló con rollback (figura 2). Antes, el 11 de noviembre, el rollback ya había actuado cuando un smoke test excedió el tiempo en una revisión en frío.

Este canary tiene límites: casi todo el tráfico que juzga es tráfico de prueba, mira errores y velocidad y no la calidad de las predicciones, y dura unos cinco minutos.

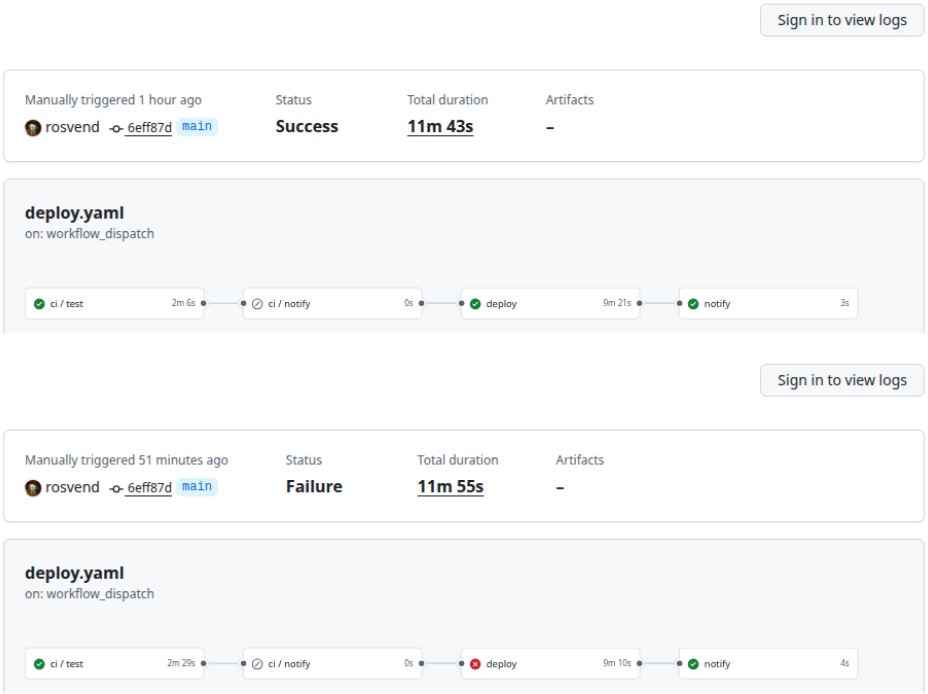


Figura 2. Deploy del 12 de noviembre de 2026. Arriba, #8: pasó el análisis y el tráfico llegó al 100 %. Abajo, #9: tope de latencia de 1 ms a propósito, el job `deploy` falla y se hace rollback; `notify` corre igual.

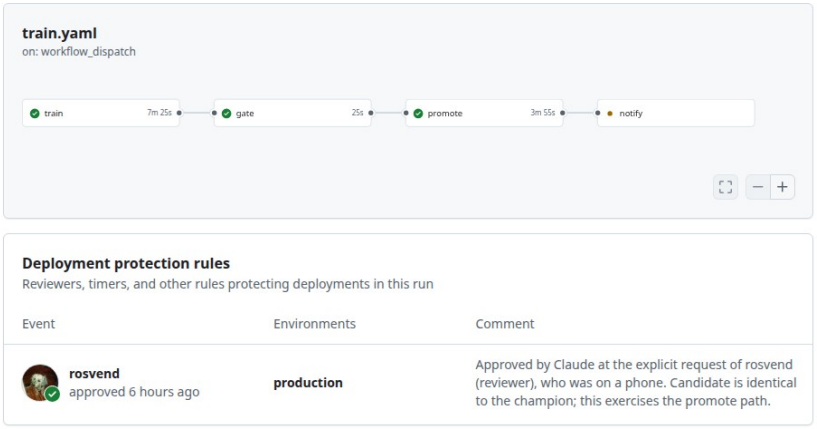


Figura 3. Train #3 (11 de noviembre de 2026): el job `promote` esperó la aprobación del entorno `production`. La ejecución figura como fallida por un «Internal server error» de GitHub; `notify` no llegó a correr y no se investigó más.

Pruebas de la API en producción. Una colección de Postman (10 solicitudes, 26 aserciones, 0 fallos) cubre predicciones, validación de entradas, rechazo de llamadas sin token y la revisión de deriva. Una prueba de carga con Locust (20 usuarios, 60 s) encontró un error real: una solicitud sin un campo opcional se registraba con `NaN`, BigQuery rechazaba la fila y la API respondía 500 en el 28 % de las solicitudes. Se corrigió (ahora se escribe `null` y un fallo del log no tumba la predicción) y el segundo ensayo no tuvo errores en 480 solicitudes, con mediana de 0,7 s y p95 de 1,6 s. La primera llamada tras un rato inactivo tardó 34 s por el arranque en frío.

Espacio para la captura	Espacio para la captura	Espacio para la captura	Espacio para la captura
Figura 4. [captura pendiente: Vertex AI, Pipelines: ejecución `loan-default-custom-run-20261111-141822` en Succeeded]	Figura 5. [captura pendiente: Vertex AI, Model Registry: `loan-default-scorer`, versiones 1 y 2, alias champion]	Figura 6. [captura pendiente: Cloud Run, `scoring-api`: pestaña Revisiones con el tráfico]	Figura 7. [captura pendiente: Facturación, Informes: últimos 30 días por servicio]

4. Rendimiento: entrenamiento en la nube frente a Python local, y AutoML

Los créditos se dividen por mes de originación: entrenan los meses más antiguos y se evalúa en los 3.050 créditos más nuevos (TEST, 101 moras). Todos los modelos se miden en esa ventana y las medianas e imputaciones se ajustan solo con entrenamiento. Se probaron tres caminos. Python local ajusta cuatro modelos en unos 2,5 minutos. Vertex Pipelines entrena los mismos tres modelos en paralelo, uno por paso en una VM `e2-highmem-2` sin GPU, con los hiperparámetros copiados de la corrida local. AutoML Tabular recibió el mismo archivo con el mínimo de presupuesto (1 node-hour) y el objetivo PR-AUC.

Tabla 2. Métricas en TEST (3.050 créditos). Entre corchetes, el intervalo bootstrap al 95 % del PR-AUC. F1 con el 15,7 % de mayor puntaje como cola de revisión. Local y Vertex dan los mismos números para los tres modelos.

Modelo	**Plataforma**	**PR-AUC [IC 95 %]**	**ROC-AUC**	**Gini**	**F1**	**Recall**	**Accuracy**
Logística	Local y Vertex	0,129 [0,076; 0,194]	0,664	0,327	0,117	0,337	83,2 %
XGBoost	Local y Vertex	0,091 [0,055; 0,150]	0,653	0,305	0,117	0,337	83,2 %
LightGBM	Local y Vertex	0,102 [0,061; 0,161]	0,640	0,280	0,114	0,327	83,1 %
Heurística	Local	0,058 [0,043; 0,086]	0,662	0,324	0,102	0,317	81,6 %
AutoML v1	Vertex AutoML	0,159 [0,098; 0,233]	0,595	0,190	0,138	0,396	83,6 %
AutoML v2	Vertex AutoML	0,215 [0,142; 0,297]	0,790	0,581	0,169	0,485	84,2 %

Tabla 3. Tiempo y costo (los costos son estimaciones con precios de lista, no una factura).

Camino	**Tiempo**	**Costo estimado**
Python local	Unos 2,5 min de ajuste	Gratis
Vertex Pipelines	Unos 6 min por ejecución; el ajuste de cada modelo toma de 0,7 a 5,8 s y el resto es arranque de VM	Menos de 0,25 USD con todos los intentos
Vertex AutoML	Unas 2 h 15 min por entrenamiento	Unos 21 USD por entrenamiento más menos de 1 USD de predicción por lotes

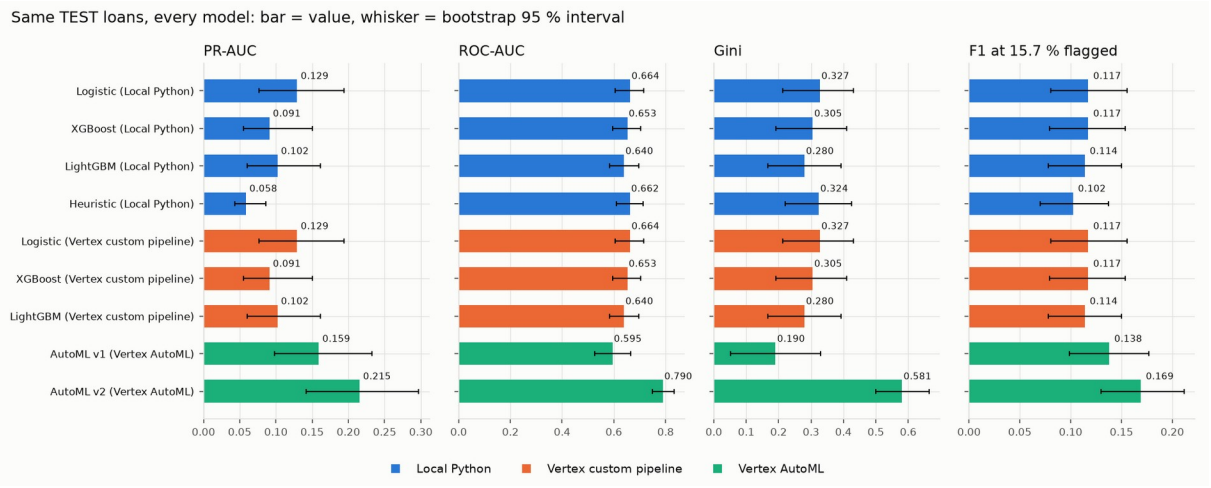


Figura 8. Métricas de ranking por modelo y plataforma en TEST (gráfico del repositorio).

Local y nube dan el mismo resultado: la diferencia máxima entre los puntajes del modelo logístico entrenado en las dos plataformas es de 8e-15, y de 0 para XGBoost y LightGBM. Pasar a la nube cambia dónde corre el entrenamiento y cuánto tarda, no lo que se aprende. Para 10.000 créditos la nube fue más lenta y más cara que el portátil, y no mejoró el modelo; se justifica con datos grandes, reentrenamientos programados y trazabilidad.

AutoML tuvo un problema práctico. La primera corrida (v1) no pudo puntuar 884 de los 3.050 créditos porque tenían celdas numéricas vacías, que AutoML rechaza al predecir, mientras que el pipeline propio las imputa. La segunda (v2) se entrenó con los datos ya imputados (medianas de TRAIN) y puntuó todo: PR-AUC 0,215 contra 0,129 de la logística, con una diferencia pareada de +0,047 a +0,126, la única mejora distinguible del ruido. Las métricas que informa Vertex (PR-AUC 0,970 y 0,973) no son comparables: parecen agrupar las dos clases, aunque la documentación no lo confirma, así que se recalcularon todas igual. No se reemplazó el campeón: es una sola ventana con 101 moras ya usada antes, v2 se corrió después de ver v1 en la misma ventana, y cuesta unos 21 USD y 2 h 15 min por entrenamiento como caja negra. La accuracy (cerca de 83 %) queda bajo la de no marcar a nadie (96,7 %), por eso la métrica principal es PR-AUC.

5. Comparación entre AWS y Google Cloud

En AWS se desplegó solo el servicio de la API, en una sesión de unos 30 minutos, todo con OpenTofu, y se destruyó al terminar (15 recursos, comprobado con la CLI). No se probó el login desde GitHub: el workflow se escribió y pasó `actionlint`, pero nunca corrió. El rol de despliegue solo se verificó con el simulador de políticas. Donde la tabla dice «documentación», no se probó.

Tabla 4. Comparación. Google Cloud: probado. AWS: probado salvo donde dice documentación.

Aspecto	**Google Cloud**	**AWS**
Configuración	OpenTofu. Hay que habilitar APIs; fallos de propagación (`SERVICE_DISABLED`) y de quota project.	OpenTofu. Sin APIs que habilitar y el apply no dio errores. La función no se crea sin imagen (apply en dos pasos) y Docker reciente exige `--provenance=false`. La cuenta no tenía límites bajos, lo que no es típico de una cuenta nueva.
API en producción	Cloud Run ejecuta el Dockerfile tal cual.	Lambda con imagen de contenedor, Lambda Web Adapter y Function URL. Necesitó otro Dockerfile, el log en `/tmp` (variable `PREDICTIONS_DB_PATH`) y vivir con un límite de inicialización de 10 s. Imagen de 1,49 GB.
Acceso privado	Token de identidad como bearer.	Function URL con `AWS_IAM`: sin firma, 403. Firmar con SigV4 pide más opciones y el token de sesión en su propia cabecera.
Canary y rollback	Revisión sin tráfico, 10 %, análisis con métricas, 100 %, rollback automático; probado tres veces.	Alias `candidate` y alias `live` con peso (10 %, luego 100 %), a mano. Rollback devolviendo el alias, en segundos. El workflow solo revierte si falla un smoke test; no hay alarma de CloudWatch ni CodeDeploy. Con 30 llamadas la versión nueva atendió entre 10 y 15 %.
Arranque en frío	34 s en la primera llamada tras un rato inactivo.	33 s la primera vez, 53 s con una versión nueva (la primera solicitud dio 502) y 9 s tras 10 min inactiva. Imágenes distintas, muestras sueltas. Mediana en caliente 0,37 s desde un portátil, sobre todo red.
Costo del servicio	Escala a cero; el costo en reposo no se midió.	Menos de 0,05 USD en total, calculado con precios de lista (Cost Explorer mostró 0 por su retraso). En reposo solo cuestan ECR y S3.
Log y deriva	BigQuery y Evidently dentro de la API.	SQLite efímero en `/tmp`: se perdió el log y con él la revisión de deriva.
Entrenamiento propio	Vertex Pipelines: tres modelos en unos 6 min, menos de 0,25 USD.	[[AWS_ENTRENAMIENTO]]
AutoML	Vertex AutoML Tabular: 1 node-hour mínimo, unas 2 h 15 min, unos 21 USD; v2 con PR-AUC 0,215.	[[AWS_ENTRENAMIENTO]]

Aspecto	Google Cloud	AWS
Registro y auditoría	Model Registry con versiones y alias; los Cloud Audit Logs no se leyeron.	Documentación: SageMaker Model Registry con estado de aprobación; CloudTrail guarda 90 días de eventos de gestión.

En el servicio, Cloud Run fue más simple: corre el contenedor sin cambios, y Lambda pidió un adaptador, un cambio de ruta de escritura y un límite de inicialización. En la infraestructura como código, AWS fue más fácil por no tener APIs que habilitar. El arranque en frío fue de decenas de segundos en ambas, con imágenes distintas. La API, su imagen y el cambio de almacenamiento del log se llevaron a AWS con cambios pequeños; el pipeline de Vertex, el código del registro, los recursos `google_*` y los comandos de despliegue no son portables.

6. Limitaciones y conclusión

La API, el pipeline y el despliegue funcionan de punta a punta en Google Cloud, con pruebas automáticas, puerta, aprobación y rollback que se usaron de verdad. Los resultados de modelos descansan en una sola ventana de 101 moras, ya usada antes, y AutoML se corrió una vez por versión. El canary juzga sobre todo tráfico de prueba. El reentrenamiento semanal rehace el mismo modelo, porque los datos son un CSV fijo. Los costos son estimaciones con precios de lista. En AWS solo se construyó el servicio, con una cuenta establecida y por una persona que ya conocía la API, así que los tiempos no son una comparación justa.